

Story Maker

Novelas personalizadas generadas por IA

Propuesta de solución | Qaracter

Portada y Contexto

Empresa presentadora: Qaracter

Cliente: [NOMBRE_CLIENTE_FICCIÓN]

Fecha: 24 de septiembre de 2026

Estudiante: Jaime Rodríguez

Problema y Cliente

¿Quién compra?

- Personas que buscan regalos personalizados y únicos
- Ocasiones especiales: bodas, cumpleaños, aniversarios, despedidas

¿Por qué las alternativas no funcionan?

- Novelas impresas genéricas: sin personalización
- Servicios manuales de escritura: tiempo y coste prohibitivo
- Al genérico: contexto limitado, falta de coherencia narrativa

Nuestra solución:

Novelas personalizadas, listas en horas, editables, en web y PDF, con validación formal

Configuración y Lectura

Flujo de entrada:

1. Entrevista estructurada → brief con personajes, trama, género, ocasión
2. Generación de 10 capítulos + portada con validación en cada paso
3. Lectura: web interactiva o PDF descargable

Edición del lector:

- Cambios puntuales en la trama o personajes
- Regeneración automática de capítulos afectados
- Revisiones incluidas en el precio (3 por novela)

Arquitectura del Harness

Actores (Agent SDK):

- Entrevistador: extrae requirements
- Planner: organiza cronología y story bible
- Writer: genera capítulos (10 en paralelo)
- Editor: valida y reescribe
- Judge: evaluación final (4 gates)

Gestión de contexto:

- Story bible: personajes, hechos, temas
- Caché de Claude API: prompt de sistema + context regenerado por turno
- Memoria: versión anterior conservada, TLA+ garantiza reintento sin duplicar

Tools:

Validación, Evaluación y Observabilidad

Cuatro tipos de validadores:

Gate	Tipo	Responsable	Criterio
1. Coherencia	LLM-as-judge	Editor	Consistencia con story bible
2. Calidad narrativa	Rubric scoring	Judge	Prosa, fluidez, impacto emocional
3. Cobertura TLA+	Formal verifier	Lean 4	Reintento sin duplicar ni perder turno
4. Seguridad	Static + guardrails	Hook	No PII, no injection, audit log

Resultados (medidos en ejecuciones reales):

Guardrails

Palabras prohibidas:

- Detección y reescritura automática
- Ejemplo: [PENDIENTE: caso de detección + reescritura]

Datos personales (PII):

- No se entrenan en briefs de clientes
- Audit log con timestamp y actor

Injection y prompt attacks:

- Sanitización de entrada en story bible
- Validación con regex + LLM en límites

Políticas:

Presupuesto y Coste

Coste unitario por novela (200/mes; `presentacion/costes.py`, tokens ⚠️ estimados hasta medir en Langfuse):

Partida	€/novela
Tokens (generación + 3 revisiones incluidas)	4,02
Infraestructura (amortizado)	0,46
Margen operativo y soporte	3,54
Coste total	8,02

Precio de venta: 29 € IVA incluido (3 revisiones) → 23,97 € netos

Margen: 15,95 € por novela (**67 %**)

Presupuesto: Escenarios de Volumen

Margen mensual neto por volumen de novelas (3 revisiones incluidas):

Vol./mes	Ingresos	Coste variable	Coste fijo	Margen	%
50	1.198 €	265 €	262 €	671 €	56 %
200	4.793 €	1.061 €	542 €	3.190 €	67 %
500	11.983 €	2.653 €	1.041 €	8.289 €	69 %

Cuello de botella: Capacidad de ejecución (1 instancia = ~500 h/mes)

Análisis de sensibilidad (detalle completo en el Anexo E):

- Tokens +50 %: margen baja a 57 % (tokens –20 %: sube a 70 %)
- 6 revisiones por novela: coste +1,77 €/novela

Demo y Cierre

Demo en vivo:

- Cambio del lector: "El protagonista debe ser ingeniero, no abogado"
- Regeneración de capítulos afectados (2–5)
- Nuevo PDF descargable en web

Demo: [PENDIENTE: captura de pantalla o vídeo demo.mp4]

Riesgos identificados:

- Capacidad: escalado a segunda instancia si > 500 novelas/mes
- Latencia: ~ 30 min por novela (medido en Langfuse)
- Coste de tokens: sensible a cambios de tarificación Anthropic

Siguientes pasos:

Contacto

Qaracter

Soluciones de storytelling personalizadas con IA

[PENDIENTE: nombre@qaracter.com]

[PENDIENTE: web]

[PENDIENTE: teléfono]

Anexo A: Arquitectura Detallada del Harness

Pipeline de ejecución:

1. **Entrevista → Brief:** Extractor (Haiku) parsea entrada libre en esquema JSON
2. **Brief → Story Bible:** Planner (Sonnet) genera personajes, hechos, temas, cronología
3. **Story Bible → Capítulos:** Writer (Sonnet) genera cap. 1–10 en paralelo con tools
4. **Capítulos → Validación:** Judge (Sonnet) evalúa coherencia, calidad, seguridad
5. **Publicación:** API genera web HTML y PDF (weasyprint)

Gestión de contexto:

- Claude API prompt caching: reutilización de system prompt + story bible
- Token budget por novela: [PENDIENTE: tokens medidos → script] (estimado hoy en `presentacion/costes.py`)
- Reintentos: si un gate falla, replanificar sin duplicar capítulos anteriores (T1 A+)

Anexo B: Especificación TLA+

Propiedades verificadas (Formal Verifier en Lean 4, CI con GitHub Actions):

INVARIANT ReintentosAcotados

$\forall t \in \text{Turnos}: \text{reintentos}[t] \leq \text{MAX_RETRIES}$

INVARIANT ReanudacionSinDuplicarNiPerder

$\forall \text{cap} \in \text{Capítulos}:$

$\text{versión}[\text{cap}] \text{ es monotónica } \wedge \text{ no hay capítulos perdidos}$

INVARIANT VersionAnteriorConservada

$\forall r \in \text{Revisiones}: \text{versión}[r-1] \text{ existe en BD}$

INVARIANT Atomicidad

$\forall \text{gate}: (\text{estado} = \text{PASS} \wedge \text{cap en BD}) \vee (\text{estado} = \text{FAIL} \wedge \text{no cambia})$

Falso positivo verificado: [PENDIENTE: contraejemplo real de Lean]

Anexo C: Tabla Completa de Resultados de Evals

[PENDIENTE: tabla de evals con scores por rubric]

Brief	Coherencia	Calidad	Coverage	Seguridad	Resultado
[PENDIENTE: B1]	[PENDIENTE]	[PENDIENTE]	[PENDIENTE]	[PENDIENTE]	PASS
[PENDIENTE: B2]	[PENDIENTE]	[PENDIENTE]	[PENDIENTE]	[PENDIENTE]	PASS

Anexo D: Esquema de la Base de Datos SQLite

Tablas principales:

```
-- Novelas
CREATE TABLE novels (
  id TEXT PRIMARY KEY,
  brief JSON NOT NULL,
  status ENUM (draft, published, archived),
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Story bibles (vectorizadas con sqlite-vec)
CREATE TABLE story_bibles (
  id TEXT PRIMARY KEY,
  novel_id TEXT REFERENCES novels(id),
  characters JSON NOT NULL,
  facts JSON NOT NULL,
  themes JSON NOT NULL,
  embedding BLOB NOT NULL -- vector de incrustación
);

-- Capítulos
CREATE TABLE chapters (
  id TEXT PRIMARY KEY,
  novel_id TEXT REFERENCES novels(id),
  chapter_num INTEGER NOT NULL,
  content TEXT NOT NULL,
  version INTEGER DEFAULT 1,
  validated_at TIMESTAMP
);

-- Evaluaciones
CREATE TABLE evaluations (
  id TEXT PRIMARY KEY,
  chapter_id TEXT REFERENCES chapters(id),
  gate_name TEXT NOT NULL,
  score FLOAT,
  passed BOOLEAN,
  result JSON NOT NULL,
  evaluated_at TIMESTAMP
);

-- Audit log
CREATE TABLE audit_log (
  id TEXT PRIMARY KEY,
  event_type TEXT NOT NULL,
```

Anexo E: Análisis de Sensibilidad Extendido

Coste variable según escenarios (200 novelas/mes; generado por `uv run presentacion/costes.py`, tokens ⚠️ estimados hasta medir en Langfuse):

Caso	€/novela	Δ respecto a base	Margen/mes (200/mes)	%
Base (Sonnet, tokens estimados)	5,31 €	—	3.190 €	67 %
Tokens +50 %	7,62 €	+2,31 €	2.728 €	57 %
Tokens –20 %	4,38 €	–0,92 €	3.375 €	70 %
6 revisiones (3 extra gratis)	7,08 €	+1,77 €	2.836 €	59 %

Anexo F: Red-Team Log

Ataques de prompt injection testeados:

Ataque	Método de detección	Resultado
Instruction override en brief del cliente	Regex + LLM boundary check	BLOCKED: reescrita sin cambiar semántica
SQL injection en story bible	Parameterización + sanitización	BLOCKED: entrada parseada con Pydantic
PII exfiltración entre clientes	Audit log + correlation check	BLOCKED: aislamiento por <code>novel_id</code>
Jailbreak via character instructions	LLM-as-judge rubric	FAILED: generación abortada, retry sin penalidad

Contramedidas en código:

Anexo G: Modelos de LLM Considerados

Comparativa de modelos candidatos:

Modelo	Entrada	Salida	Contexto	Elección	Motivo
Claude Haiku 4.5	1 \$/M	5 \$/M	200k	Extractor, revisor	Bajo coste, comprensión suficiente
Claude Sonnet 5	2 \$/M	10 \$/M	200k	Planner, Writer, Judge	Mejor en creatividad y consistencia
Claude Opus 5.5	4 \$/M	20 \$/M	200k	No elegido	ROI negativo: +50% coste, +2% score
GPT-4o	5 \$/M	15 \$/M	200k	Descartado	No compatible con Agent SDK, más latencia

Resultado: Sonnet 5 es el óptimo costo-calidad para generación; Haiku para extracción

Anexo H: Capturas de Langfuse

Traza real de una novela (de Langfuse Cloud UE, plan Hobby):

Ejemplo de observaciones capturadas:

```
{
  "trace_id": "uuid-12345",
  "project": "story-maker-demo",
  "tags": ["production", "brief-001"],
  "observations": [
    {
      "name": "entrevistador",
      "type": "generation",
      "model": "claude-haiku-4-5",
      "input_tokens": 1250,
      "output_tokens": 350,
      "latency_ms": 3200,
      "total_cost_usd": 0.012
    },
    {
      "name": "planner",
      "type": "generation",
      "model": "claude-sonnet-5",
      "input_tokens": 8900,
      "output_tokens": 2100,
      "latency_ms": 8500,
      "total_cost_usd": 0.045
    },
    {
      "name": "writer",
      "type": "generation",
      "model": "claude-sonnet-5",
      "input_tokens": 65000,
      "output_tokens": 18000,
      "latency_ms": 45000,
      "total_cost_usd": 0.210
    },
    {
      "name": "judge",
      "type": "generation",
      "model": "claude-sonnet-5"
```

Fin de la presentación

Preguntas técnicas: 5 minutos

Gracias por su atención.

© 2026 Qaracter | Propuesta confidencial